

Causal Attention、Logit-Lens 与 Jacobian-Lens

从序列位置交互到模型内部概念读出

目标：把三者统一到同一个计算图与线性读出框架中。

本课件解决的三个问题

1. **Causal attention** 如何计算？为什么过去位置的 hidden state 会影响未来位置的输出？
2. **Logit-lens** 在数学上读的是什​​么？为什么它可以把中间层 residual stream 直接解码成 token？
3. **Jacobian-lens** 改进在哪里？它如何用平均 Jacobian 修正跨层表征几何，并读出未说出口的中间概念？

统一视角

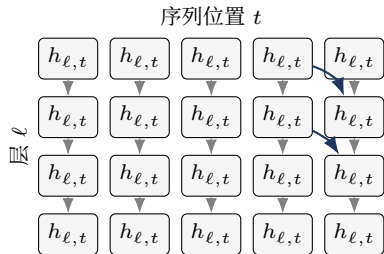
三者都在分析同一个对象：decoder-only Transformer 中每一层、每个 token 位置的 residual stream

$$h_{\ell,t} \in \mathbb{R}^{d_{\text{model}}}.$$

区别在于：attention 描述它如何传播，lens 描述我们如何读它。

Causal Attention 的计算机制

Transformer 的二维计算图：层维度与位置维度



两个方向

垂直方向：

$$h_{\ell, t} \rightarrow h_{\ell+1, t} \rightarrow \dots \rightarrow h_{L, t}$$

横向传播：

$$h_{\ell, t} \rightarrow h_{\ell+1, t'} \quad (t' \geq t)$$

横向传播来自 causal self-attention：后面位置可以读取前面位置。

Notation: 一条 prompt 的 residual stream

设输入 token 序列为

$$x_0, x_1, \dots, x_T.$$

第 ℓ 层输出是一整排 residual stream:

$$H_\ell = (h_{\ell,0}, h_{\ell,1}, \dots, h_{\ell,T}), \quad h_{\ell,t} \in \mathbb{R}^{d_{\text{model}}}.$$

关键对象

- ▶ ℓ : 层编号, 表示网络深度。
- ▶ t : 源 token 位置, 例如 “France” 的位置。
- ▶ t' : 目标 token 位置, 例如 “is” 的位置。
- ▶ L : 最终层, 最终 logits 从 $h_{L,t}$ 经 unembedding 读出。

例子: 在 “The capital of France is” 中, 可令 $t = 3$ 对应 “France”, $t' = 4$ 对应 “is”。

单头 causal self-attention: 从 H_ℓ 到 $H_{\ell+1}$

对第 ℓ 层输入 $h_{\ell,t}$, 先得到 query、key、value:

$$q_t = W_Q h_{\ell,t}, \quad k_i = W_K h_{\ell,i}, \quad v_i = W_V h_{\ell,i}. \quad (1.1)$$

位置 t' 对历史位置 $i \leq t'$ 的注意力权重为

$$\alpha_{t',i} = \frac{\exp(q_{t'}^\top k_i / \sqrt{d_k})}{\sum_{j \leq t'} \exp(q_{t'}^\top k_j / \sqrt{d_k})}. \quad (1.2)$$

attention 输出:

$$a_{t'} = \sum_{i \leq t'} \alpha_{t',i} v_i. \quad (1.3)$$

Causal mask 的作用

t' 只能看 $i \leq t'$, 不能看 $i > t'$ 。因此后面的 token 可以读取前面的 token; 前面的 token 不能读取后面的 token。

一层更新：未来位置如何读取过去位置

忽略多头、输出矩阵和归一化细节，一层更新可写成

$$h_{\ell+1,t'} = h_{\ell,t'} + \sum_{i \leq t'} \alpha_{t',i} W_V h_{\ell,i} + \text{MLP}_{\ell}(\dots). \quad (1.4)$$

当 $i = t < t'$ 时，右侧有一项

$$\alpha_{t',t} W_V h_{\ell,t}. \quad (1.5)$$

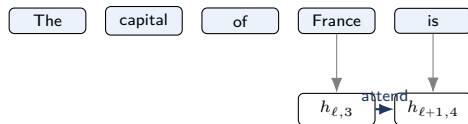
因此

$$h_{\ell,t} \text{ 会影响 } h_{\ell+1,t'} \quad (t' \geq t). \quad (1.6)$$

精确定义

不是 $h_{\ell,t}$ 与 $h_{\ell,t'}$ 在同一层互相融合；而是计算 $h_{\ell+1,t'}$ 时，位置 t' 使用了历史位置 t 的 value 信息。

例子：“France” 如何影响 “is” 后的预测



在某层中，“is” 位置可以通过 attention 读取 “France” 位置：

$$h_{\ell+1,4} \ni \alpha_{4,3} W_V h_{\ell,3}.$$

若把 $h_{\ell,3}$ 轻微推向 “Germany” 方向，最终 $h_{L,4}$ 的预测可能从

Paris

转向

Berlin.

这就是跨位置因果路径。

多层传播：从局部扰动到最终层表示

在第 ℓ 层、位置 t 加一个小扰动：

$$h_{\ell,t} \leftarrow h_{\ell,t} + \delta h.$$

经过后续层传播后，最终层目标位置 t' 改变为

$$h_{L,t'} \leftarrow h_{L,t'} + \delta h_{L,t'}.$$

一阶线性近似：

$$\delta h_{L,t'} \approx \frac{\partial h_{L,t'}}{\partial h_{\ell,t}} \delta h. \quad (1.7)$$

其中

$$A_{\ell,t,t'} = \frac{\partial h_{L,t'}}{\partial h_{\ell,t}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}} \quad (1.8)$$

就是一个 prompt、一对位置 (t, t') 下的局部 Jacobian。

为什么必须允许 $t' \geq t$?

同位置传播

$$\frac{\partial h_{L,t}}{\partial h_{\ell,t}}$$

表示 token 自己沿层方向向下传播。

未来位置传播

$$\frac{\partial h_{L,t'}}{\partial h_{\ell,t}}, \quad t' > t$$

表示过去位置的信息被后面 token 通过 attention 读取。

causal mask 的限制

标准 decoder-only Transformer 中:

$$t' < t \Rightarrow \frac{\partial h_{L,t'}}{\partial h_{\ell,t}} \approx 0.$$

因为前面位置不能读取未来位置。

对 J-lens 的意义

J-lens 不只看“这个位置自己会变成什么”，还看“这个位置的信息会如何影响后续位置的 verbalization”。

小结: causal attention 的计算机制

1. 每层输入是一组位置向量 $h_{\ell,0:T}$ 。
2. 每个目标位置 t' 生成 query, 并对所有历史位置 $i \leq t'$ 的 key 做匹配。
3. value 加权和被写入 $h_{\ell+1,t'}$ 。
4. 因此, 过去位置 t 的表示可以影响未来位置 t' 的下一层表示。
5. 多层堆叠后, 这种影响可由

$$\frac{\partial h_{L,t'}}{\partial h_{\ell,t}}$$

量化。

对后续 lens 的铺垫

Logit-lens 直接读 $h_{\ell,t}$; Jacobian-lens 读的是 $h_{\ell,t}$ 经过平均后续传播后的效果。

Logit-Lens: 直接用输出头读取中间层

最终输出头：从最终 residual 到 logits

在最终层 L ，模型对位置 t 的下一个 token 预测为

$$z_t = W_U \text{norm}(h_{L,t}), \quad (2.1)$$

$$p_t = \text{softmax}(z_t), \quad (2.2)$$

其中

$$W_U \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$$

是 unembedding 矩阵， $|\mathcal{V}|$ 是词表大小。

某个 token 的 logit

若 $w_y^\top$ 是 W_U 中 token y 对应的行，则

$$z_t(y) = w_y^\top \text{norm}(h_{L,t}).$$

输出层读出的本质是：最终 residual 与词表方向的内积。

Logit-lens 的定义

Logit-lens 把中间层 $h_{\ell,t}$ 当成最终层 residual, 直接套用最终输出头:

$$z_{\ell,t}^{\text{LL}} = W_U \text{norm}(h_{\ell,t}), \quad (2.3)$$

$$p_{\ell,t}^{\text{LL}} = \text{softmax}(z_{\ell,t}^{\text{LL}}). \quad (2.4)$$

忽略归一化时:

$$z_{\ell,t}^{\text{LL}}(y) = w_y^\top h_{\ell,t}. \quad (2.5)$$

核心问题

第 ℓ 层的 hidden state 是否已经沿最终输出词表方向编码了某个 token 或概念?

Logit-lens 的隐含假设

Logit-lens 等价于假设所有层共享同一套输出坐标系：

$h_{\ell,t}$ 可以被 W_U 直接解释。

合理的区域

后几层通常更接近最终输出空间，logit-lens 往往能读到下一个 token 的形成过程。

容易失败的区域

早中层的表征可能还没有被后续层旋转、压缩、重排到输出空间，因此直接用 W_U 可能读出噪声。

Logit-lens: $h_{\ell,t} \xrightarrow{W_U} \text{token logits.}$

Logit-lens 的实现流程

1. 输入 prompt, 运行一次普通 forward pass。
2. 保存每层 residual stream: $h_{\ell,t}$ 。
3. 对选定层 ℓ 和位置 t , 计算:

$$z_{\ell,t}^{\text{LL}} = W_U \text{norm}(h_{\ell,t}).$$

4. 对 logits 排序, 得到 top- k token。
5. 画 rank / logit / probability 随层变化的曲线。

伪代码

```
hidden = model(prompt, output_hidden_states=True)
h = hidden[layer][position]
logits = unembed(final_norm(h))
top_tokens = sort(logits)
```

Logit-lens 可以回答什么问题？

适合的问题

- ▶ 目标答案 token 从哪一层开始出现？
- ▶ 正确答案和干扰答案的 logit gap 如何随层变化？
- ▶ 模型最终预测是否已在中间层形成？

不适合的问题

- ▶ 早层里的抽象中间概念是什么？
- ▶ 未说出口的推理变量是否存在？
- ▶ 某个概念是否以可被未来 verbalize 的格式存在？

限制

Logit-lens 是非常有用的低成本近似，但它没有显式考虑 $h_{\ell,t}$ 到 $h_{L,t'}$ 的后续传播。

Jacobian-Lens: 用平均后续传播修正
读出

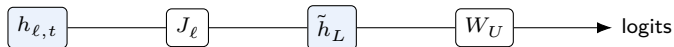
J-lens 的动机：中间层坐标系不等于最终层坐标系

J-lens 的出发点是：第 ℓ 层的一个方向未必能被最终 unembedding 直接读取，但它经过后续层传播后，可能稳定地变成某个可 verbalize 的概念方向。

$$h_{\ell,t} \xrightarrow{\text{后续 Transformer 层}} h_{L,t'} \xrightarrow{W_U} \text{token logits.} \quad (3.1)$$

关键改动

Logit-lens 用恒等映射代替后续层；J-lens 用平均 Jacobian 近似后续层的线性效应。



单个 prompt 下的局部 Jacobian

对给定 prompt、源位置 t 、目标位置 $t' \geq t$ ，定义

$$A_{\ell,t,t'} = \frac{\partial h_{L,t'}}{\partial h_{\ell,t}}. \quad (3.2)$$

如果对 $h_{\ell,t}$ 加小扰动 δh ，则

$$\delta h_{L,t'} \approx A_{\ell,t,t'} \delta h. \quad (3.3)$$

再接上 unembedding：

$$\delta z_{t'} \approx W_U A_{\ell,t,t'} \delta h. \quad (3.4)$$

解释

$A_{\ell,t,t'}$ 衡量：第 ℓ 层、位置 t 的某个方向，会如何影响最终层、位置 t' 的输出倾向。

为什么要平均 Jacobian?

单个 prompt 的 Jacobian 太具体，会混合两类因素：

1. 某个概念在当前上下文中的特殊用途。
2. 这个概念一般性地进入 verbalizable format 的趋势。

因此 J-lens 对 prompt、源位置、后续位置做平均：

$$J_\ell = \mathbb{E}_{\text{prompt}, t, t' \geq t} \left[\frac{\partial h_{L, t'}}{\partial h_{\ell, t}} \right]. \quad (3.5)$$

结果

每一层得到一个矩阵

$$J_\ell \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}},$$

作为“第 ℓ 层 residual space 到最终层 residual space”的平均线性地图。[1]

J-lens 的读出公式

给定中间层 activation $h_{\ell,t}$, J-lens 先用 J_ℓ 映射到最终层坐标系:

$$\tilde{h}_L = J_\ell h_{\ell,t}.$$

再用正常 unembedding:

$$z_{\ell,t}^J = W_U \text{norm}(J_\ell h_{\ell,t}), \quad (3.6)$$

$$p_{\ell,t}^J = \text{softmax}(z_{\ell,t}^J). \quad (3.7)$$

忽略归一化, token y 的 logit 为

$$z_{\ell,t}^J(y) = w_y^\top J_\ell h_{\ell,t} = (J_\ell^\top w_y)^\top h_{\ell,t}. \quad (3.8)$$

J-lens vector: 把最终 token 方向拉回中间层

由

$$z_{\ell,t}^J(y) = (J_{\ell}^{\top} w_y)^{\top} h_{\ell,t}$$

可知, token y 在第 ℓ 层的 J-lens 方向是

$$v_{\ell,y}^J = J_{\ell}^{\top} w_y. \quad (3.9)$$

对比 logit-lens:

$$v_y^{\text{LL}} = w_y. \quad (3.10)$$

一句话

Logit-lens 用最终输出方向 w_y 直接读中间层; J-lens 先把 w_y 通过 $J_{\ell}^{\top}$ 拉回到第 ℓ 层, 得到该层真正对应的可 verbalize 概念方向。

J-lens 的离线与在线流程

离线构造

1. 采样 calibration prompts。
2. 对每个 $\ell, t, t' \geq t$ 计算 Jacobian。
3. 对 prompt、位置、后续位置求平均。
4. 存储每层矩阵 J_ℓ 。

在线读取

1. 对新 prompt 做一次 forward。
2. 取某层某位置 $h_{\ell,t}$ 。
3. 计算 $J_\ell h_{\ell,t}$ 。
4. 用 W_U 读 top- k token。

代价

Logit-lens 几乎只需 forward；J-lens 需要离线估计 Jacobian，成本高得多，但读出阶段仍是线性映射。

完整例子：“France” 位置的 J-lens 读出

Prompt:

The capital of France is

选择 $\ell = 12$, $t = 3$, 即 “France” 位置的 activation:

$$h_{12,3}.$$

J-lens 计算:

$$z_{12,3}^J = W_U \text{norm}(J_{12} h_{12,3}). \quad (3.11)$$

可能的 top tokens:

France, Paris, capital, city, Europe.

解释

这些 token 不一定是当前位置马上要输出的词，而是表示该 activation 经平均后续传播后，容易使模型在当前或未来位置 verbalize 的概念。

Self-only 与 Future/Broadcast 成分

J-lens 中的平均可以拆成两类:

Self-only

只看 $t' = t$:

$$\mathbb{E} \left[\frac{\partial h_{L,t}}{\partial h_{\ell,t}} \right].$$

更接近“当前位置自己的垂直传播”。

Future / Broadcast

只看 $t' > t$:

$$\mathbb{E} \left[\frac{\partial h_{L,t'}}{\partial h_{\ell,t}} \right].$$

更接近“该位置信息被后续位置读取并广播”。

认知解释

如果一个概念不只影响自身位置，而能被后续多个位置读取和使用，它就更像一种可供下游计算访问的 workspace 内容。

从 J-lens vectors 到 J-space

每层有一组 token-indexed J-lens vectors:

$$\{v_{\ell,y}^J : y \in \mathcal{V}\}.$$

由于 $|\mathcal{V}| > d_{\text{model}}$, 这些向量不是普通意义上的低维基底。J-space 通常被理解为 activation 中可由少数 J-lens vectors 稀疏表示的成分:

$$h_{\ell,t}^J \approx \sum_{y \in S} c_y v_{\ell,y}^J, \quad c_y \geq 0, \quad |S| \leq k. \quad (3.12)$$

直观含义

J-space 是 residual stream 中可以被分解为少数几个“可说出的概念”的部分; non-J-space 是其余不以这种格式存在的信息。

用 J-lens 读概念与写概念

Reading

- ▶ 读 top- k tokens。
- ▶ 追踪目标 token 的 rank/logit。
- ▶ 用稀疏分解估计 active J-space contents。

Writing

- ▶ Steering: $h \leftarrow h + \alpha v_{\ell,y}^J$ 。
- ▶ Ablation: 移除某个概念方向。
- ▶ Swap: 交换两个 lens coordinates。

$$h_{\text{patched}} = h + V(\sigma(V^\dagger h) - V^\dagger h), \quad (3.13)$$

其中 $V = [v_s \ v_t]$, σ 交换 source/target 两个概念坐标。[1]

数学差别与结果差别

核心数学差别：是否校正后续层

Logit-lens

$$z_{\ell,t}^{\text{LL}} = W_U \text{norm}(h_{\ell,t}).$$

等价于令

$$J_{\ell} = I.$$

假设中间层已经在最终输出坐标系中。

Jacobian-lens

$$z_{\ell,t}^{\text{J}} = W_U \text{norm}(J_{\ell} h_{\ell,t}).$$

其中

$$J_{\ell} = \mathbb{E} \left[\frac{\partial h_{L,t'}}{\partial h_{\ell,t}} \right].$$

显式近似后续层的平均传播。

Logit-lens 是 J-lens 在 $J_{\ell} = I$ 下的特殊近似。

读出方向的差别

忽略归一化时，二者都可以看作 inner product probe。

Logit-lens token 方向

$$v_y^{\text{LL}} = w_y.$$

读的是最终输出头中 token y 的方向。

J-lens token 方向

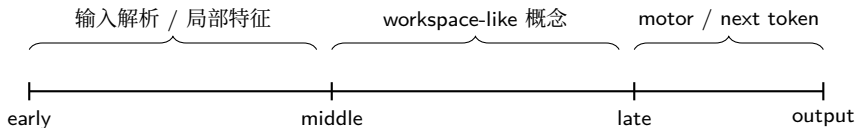
$$v_{\ell,y}^{\text{J}} = J_{\ell}^{\top} w_y.$$

读的是“会经后续层变成 token y 的第 ℓ 层方向”。

结果含义

Logit-lens 更像直接问“现在是否已经像输出 token”；J-lens 更像问“这个中间方向是否会被后续网络转化为可 verbalize 的 token”。

不同层段的结果差别



早期层

Logit-lens 常噪声较大; J-lens 也可能不稳定, 因为 coherent content 尚未形成。

中间层

J-lens 更容易读出未说出口的中间概念; logit-lens 有时也有效, 但可靠性较低。

后期层

二者通常更接近; 读出逐渐转向即将输出的 token。

结果上的三种典型差异

1. **早中层可解释性：** J-lens 往往比 logit-lens 更能恢复有意义的中间内容。
2. **任务目标：** logit-lens 更接近“目标答案何时进入输出空间”；J-lens 更接近“哪些概念处于可被未来 verbalize 的格式”。
3. **干预方向：** logit-lens 的 w_y 更适合后层 steering；J-lens 的 $J_\ell^\top w_y$ 更适合中间层 concept swap / ablation。

重要提醒

J-lens 不是为了最好预测 next token 而设计；它的目标是暴露和干预模型内部的可报告中间表征。

计算成本对比

方法	主要计算	成本
Logit-lens	forward + $W_U h_{\ell,t}$	低
Tuned lens	训练每层线性 map	中
Jacobian-lens	估计 $\mathbb{E}[\partial h_L / \partial h_\ell]$	高

实践含义

若目标是快速诊断目标 token 的层级演化，先用 logit-lens。若目标是严肃分析中间概念、未输出推理、可 verbalize workspace，才值得引入 J-lens 或其简化版本。

在记忆 / PI / RPI 实验中的用法

Logit-lens 优先场景

- ▶ 目标 value token 的 logit 何时衰减?
- ▶ 干扰 value 是否在后层上升?
- ▶ 正确答案与 previous answer 的 margin 如何变化?

J-lens 更适合场景

- ▶ key、value、category 是否在输出前被内部持有?
- ▶ previous answer 是否进入可 verbalize workspace?
- ▶ release 后新 category 是否替换旧 category?

建议

先把 logit-lens 做成稳定 pipeline；再尝试简化 J-lens，例如少数层、少数位置、只估计目标 token 的 VJP，而不是完整 $d \times d$ 矩阵。

可实现的简化版 J-lens

完整 J-lens 昂贵。实验上可先做三个近似：

1. **少层采样**：只在 4–6 个代表层估计 J_ℓ 。
2. **只看目标位置**：先用 self-only 或最后 query 位置，而不是所有 $t' \geq t$ 。
3. **token-specific VJP**：对目标 token y 只计算

$$\nabla_{h_{\ell,t}} z_{t'}(y) \approx A_{\ell,t,t'}^\top w_y,$$

避免显式存储完整 J_ℓ 。

代价-解释性折中

这种简化版不能完全复现论文中的 J-space，但足以测试“目标概念是否在中间层具有未来输出因果效应”。

常见误解澄清

误解一：attention 是同层双向交互

不是。位置 t' 在下一层读取历史位置 t ；过去位置不读取未来位置。

误解二：logit-lens 读到的就是模型真实想法

不是。它只是用最终输出头直接投影中间层，可能受坐标 mismatch 影响。

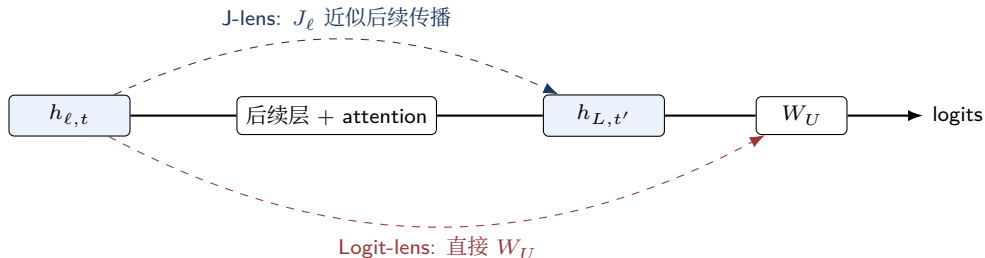
误解三：J-lens top token 等于模型马上要输出的 token

不是。J-lens 更接近读“可被未来说出的内部概念”，特别是在中间层。

误解四：J-space 是一个固定低维线性子空间

不是。它更接近由少数 J-lens vectors 稀疏非负组合定义的 activation 成分。

总括：一张图理解三者



- ▶ Causal attention: 解释 $h_{\ell,t}$ 如何影响 $h_{L,t'}$ 。
- ▶ Logit-lens: 跳过后续层, 直接问 $h_{\ell,t}$ 像什么 token。
- ▶ J-lens: 用平均 Jacobian 近似后续层, 再问它会推动什么 token。

核心结论

1. Causal attention 让过去位置的信息进入未来位置，因此中间层 activation 的影响可以跨 token 传播。
2. Logit-lens 是最简单的中间层解码：

$$W_U \text{norm}(h_{\ell,t}).$$

它便宜、直观，但假设跨层坐标一致。

3. J-lens 用

$$J_{\ell} = \mathbb{E} \left[\frac{\partial h_{L,t'}}{\partial h_{\ell,t}} \right]$$

修正后续传播，读出更接近可 verbalize 的中间概念。

4. 结果上，logit-lens 更适合看输出倾向形成；J-lens 更适合看 workspace-like 中间表征。

参考文献 I

1. Gurnee, W. *et al.* Verbalizable Representations Form a Global Workspace in Language Models. [Transformer Circuits Thread](#), Anthropic (2026).